

ATIVIDADE: LISTAS DE TAREFAS

1. Crie uma classe de testes para uma classe Gerenciadora de Tarefas com as seguintes funcionalidades:
 - Adicionar, remover e marcar tarefas como concluídas;
 - Filtrar tarefas por status;
 - Ordenar tarefas por data de criação.

```
public class GerenciadorTarefas { no usages
    private List<Tarefa> tarefas = new ArrayList<>(); 10 usages

    public boolean adicionar(Tarefa tarefa) { no usages
        if (tarefas.contains(tarefa)) {
            return false;
        }
        return tarefas.add(tarefa);
    }

    public boolean remover(String descricao) { no usages
        return tarefas.removeIf(Tarefa t -> t.getDescricao().equals(descricao));
    }

    public void marcarComoConcluida(String descricao) { no usages
        for (Tarefa t : tarefas) {
            if (t.getDescricao().equals(descricao)) {
                t.setConcluida(true);
                break;
            }
        }
    }
}
```

```

public void marcarTodasComoConcluidas() { no usages
    tarefas.forEach( Tarefa t -> t.setConcluida(true));
}

public List<Tarefa> filtrarPorStatus(boolean concluida) { no usages
    return tarefas.stream()
        .filter( Tarefa t -> t.isConcluida() == concluida)
        .collect(Collectors.toList());
}

public List<Tarefa> ordenarPorDataCriacao() { no usages
    return tarefas.stream()
        .sorted(Comparator.comparing(Tarefa::getDataCriacao))
        .collect(Collectors.toList());
}

public void salvarEmArquivo(String caminho) throws IOException { no usages
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(caminho))) {
        oos.writeObject(tarefas);
    }
}

```

```

@SuppressWarnings("unchecked") no usages
public void carregarDeArquivo(String caminho) throws IOException, ClassNotFoundException {
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(caminho))) {
        tarefas = (List<Tarefa>) ois.readObject();
    }
}

public List<Tarefa> getTarefas() { no usages
    return new ArrayList<>(tarefas);
}
}

```

2. Considere os seguintes cenários de teste:

- Adicionar e remover tarefas duplicadas;
- Marcar todas as tarefas como concluídas;
- Filtrar tarefas por diferentes critérios;
- Salvar e carregar as tarefas em arquivos.

```

public class GerenciadorTarefasTest {
    private GerenciadorTarefas gerenciador; 21 usages
    private final String CAMINHO_TESTE = "tarefas_teste.dat";

    @BeforeEach
    void setup() {
        gerenciador = new GerenciadorTarefas();
    }

    @AfterEach
    void tearDown() {
        File arquivo = new File(CAMINHO_TESTE);
        if (arquivo.exists()) {
            arquivo.delete();
        }
    }
}

```

```

@Test
@DisplayName("Deve adicionar e remover tarefas corretamente")
void testarAdicaoRemocao() {
    Tarefa t1 = new Tarefa(descricao: "Tarefa 1");
    assertTrue(gerenciador.adicionar(t1));
    assertEquals(expected: 1, gerenciador.getTarefas().size());

    assertTrue(gerenciador.remover(descricao: "Tarefa 1"));
    assertEquals(expected: 0, gerenciador.getTarefas().size());
}

```

```

@Test
@DisplayName("Não deve permitir adicionar tarefas duplicadas")
void testarDuplicados() {
    gerenciador.adicionar(new Tarefa(descricao: "Estudar JUnit"));
    assertFalse(gerenciador.adicionar(new Tarefa(descricao: "Estudar JUnit")));
    assertEquals(expected: 1, gerenciador.getTarefas().size());
}

```

```

@Test
@DisplayName("Deve marcar tarefas como concluídas")
void testarConclusao() {
    gerenciador.adicionar(new Tarefa( descricao: "Tarefa A"));
    gerenciador.adicionar(new Tarefa( descricao: "Tarefa B"));

    gerenciador.marcarComoConcluida( descricao: "Tarefa A");
    assertTrue(gerenciador.getTarefas().get(0).isConcluida());

    gerenciador.marcarTodasComoConcluidas();
    assertTrue(gerenciador.getTarefas().get(1).isConcluida());
}

@Test
@DisplayName("Deve filtrar tarefas por status")
void testarFiltro() {
    gerenciador.adicionar(new Tarefa( descricao: "T1"));
    gerenciador.adicionar(new Tarefa( descricao: "T2"));
    gerenciador.marcarComoConcluida( descricao: "T1");

    assertEquals( expected: 1, gerenciador.filtrarPorStatus( concluida: true).size());
    assertEquals( expected: 1, gerenciador.filtrarPorStatus( concluida: false).size());
}

```

```

@Test
@DisplayName("Deve salvar e carregar tarefas de um arquivo")
void testarPersistencia() throws Exception {
    gerenciador.adicionar(new Tarefa( descricao: "Tarefa Persistida"));
    gerenciador.salvarEmArquivo(CAMINHO_TESTE);

    GerenciadorTarefas novoGerenciador = new GerenciadorTarefas();
    novoGerenciador.carregarDeArquivo(CAMINHO_TESTE);

    assertEquals( expected: 1, novoGerenciador.getTarefas().size());
    assertEquals( expected: "Tarefa Persistida", novoGerenciador.getTarefas().get(0).getDescricao());
}
}

```